**Large Scale Social and Complex Networks: Design and Algorithms**
**ECE 232E — Summer 2026**

Prof. Vwani Roychowdhury UCLA Department of Electrical and Computer Engineering

---

# Course Project
## Structured Memory Networks and Reasoning Inference Chain Retrieval

Based on PANINI: Continual Learning in Token Space via Structured Memory

**150 points**

Due Saturday, August 15, 2026 at 11:59 PM Pacific Time

---

## 1 The project: inherit, understand, and operate a structured memory

Imagine joining a team after its write-time system has processed a collection of documents. The original passages are no longer the primary memory. Instead, every document has become a Generative Semantic Workspace (GSW): a small heterogeneous network of entities, verb phrases, roles, states, and grounded question–answer relations. Your task is to make this accumulated memory answer new multi-hop questions without flattening it back into long chunks.

The project follows the life of one system from handoff to deployment. You will first determine what was written and whether separate workspaces can be viewed as a meaningful global network. You will then receive a user question, turn it into an executable dependency graph, retrieve one piece of evidence at a time, and connect those pieces using Reasoning Inference Chain Retrieval (RICR). Finally, you will freeze the design on 2WikiMultiHopQA, move it unchanged to MuSiQue, and hand the complete reproducible system to another team.

At each stage you must do two things: implement the mechanism and explain in your own words what its output means. A correct number without an interpretation is incomplete, because network properties can be artifacts of entity reconciliation, retrieval scores can hide mismatched candidates, and a correct final answer can still be supported by a broken reasoning chain.

## 2 What your team receives

**Required reading.** Read Sections 1–4 and Algorithm 1 of PANINI: Continual Learning in Token Space via Structured Memory. The GSW representation, dual indexing, decomposition notation, chain scoring, and RICR pruning rule in the paper are part of the project specification.

**Student repository.** The self-contained starter repository is:

`https://github.com/YigitTurali/panini-course-project`

The Colab notebook, artifact loaders, graph utilities, metrics, embeddings, indices, and model configuration are supplied. Students do **not** need access to the research codebase.

**Models.** Use the model identifiers in `models/model_config.json`. The required free-tier configuration uses:

- GSW-QA-Decomposer-Qwen3-4B for decomposition;
- supplied Qwen3-Embedding-8B corpus and fixed-query embeddings;
- Qwen3-Reranker-8B in 4-bit mode for reranking; and
- Qwen3-4B in 4-bit mode for evidence-grounded final answers.

The 8B decomposer is optional and receives no automatic accuracy advantage. Do not keep the decomposer, reranker, query encoder, and answer model resident on the GPU simultaneously. Save outputs to JSONL, unload the current model, clear GPU memory, and then begin the next stage.

**Compute policy.** The required path must run on a free Google Colab GPU. No paid API is required or permitted for the required results. Corpus GSWs, embeddings, and indices are supplied; regenerating them is outside the project scope.

## 3 Stage I: The memory arrives

Your team is taking over a PANINI memory after its write-time workers have finished. Two collections arrive: one built from 2WikiMultiHopQA and one from MuSiQue. They were produced independently, but they use the same directory contract and stable-ID scheme so that one implementation can operate on both. Each package contains 100 questions and only the document-level GSWs, metadata, embeddings, and indices needed for this project.

| Dataset | Development | Held out | Total | Required stratification |
|---|---|---|---|---|
| 2WikiMultiHopQA | 80 | 20 | 100 | 25 bridge-comparison, 25 comparison, 25 compositional, and 25 inference questions; each category has 20 development and 5 held-out questions. |
| MuSiQue | 80 | 20 | 100 | 50 two-hop, 30 three-hop, and 20 four-hop questions; development/held-out counts are respectively 40/10, 24/6, and 16/4. |

Table 1: Required course subsets.

- The **development split** includes questions, answers, aliases, supporting evidence, and context-document IDs. It may be used for debugging and tables in the report.
- The **held-out split** includes questions and document IDs but withholds answers and supporting evidence. Do not manually search for, reconstruct, or import these labels.
- Select algorithms and default hyperparameters using only the 2Wiki development split. Freeze them before running MuSiQue. MuSiQue is the required cross-dataset transfer evaluation.
- Run the final frozen system once on all 200 questions. Report metrics locally for the 160 development questions; the teaching staff will score the 40 held-out predictions.

MuSiQue is distributed under CC BY 4.0; 2WikiMultiHopQA is distributed under Apache 2.0. Retain the dataset IDs and attribution files in all derived outputs.

### 3.1 Question 1: Understand the two data packages [6 points]

Imagine that the write-time stage of PANINI has just finished processing two collections of documents. You did not run that stage: you inherit its document-level GSWs, flattened entity and QA tables, embeddings, and indices. Before treating these files as one memory, you must establish what one row means and which identifiers remain valid when records from many documents are combined. A retrieval result that points to the wrong embedding row can look plausible while silently invalidating every later experiment.

Load both packages with `CoursePackage`. Use `entity_uid` and `qa_uid` as identifiers rather than list positions. Check that every embedding row has exactly one matching ID, IDs are unique, and held-out files contain inputs but no grading labels. This audit becomes the trusted boundary for all later questions.

(a) Produce a table containing, for each dataset and split, the number of questions, unique documents, GSW files, entities, QA records, and decomposition-validation examples.

(b) Show one question, entity, and QA record. Explain the stable ID and every field that your implementation consumes.

(c) Add an assertion that held-out records contain no answer-label fields: `answer`, `answer_aliases`, `supporting_facts`, or `evidences`.

(d) Run `pytest -q tests`. Record which tests pass and which RICR tests are skipped before you begin the implementation.

(e) In your own words, explain why stable IDs are necessary when graph records, embedding rows, and retrieval results are stored in different files. Give one plausible failure that a row-count check alone would not detect.

**Required output.** One dataset table, three short schema examples, the no-leakage assertion, the initial test summary, and a 150–200 word explanation.

**Point allocation.** Dataset table (1), schema explanation (1), automated checks and starter-test run (2), and own-words explanation (2).

## 4 Stage II: The local workspaces become a network

The package audit tells you that every record can be traced back to its document. Now open those documents conceptually. A GSW is not a conventional knowledge graph with one globally resolved node for every real-world object. It is a local account of an experience: entity, space/time, and verb-phrase nodes are tied together by grounded QA answers. Combining files therefore creates two different scientific objects. The first is the network that was actually written. The second is a network inferred by deciding which local occurrences should share a global identity.

Begin with the **native directed multigraph**. Each local record keeps a document-namespaced UID. Each grounded answer creates a directed edge from its verb-phrase node to the answer node, keyed by stable QA ID. From this graph, form an **unreconciled entity projection**: two document-local answer occurrences are adjacent when they belong to the same verb-phrase neighborhood, and repeated neighborhoods increment edge weight.

Only then propose cross-document identities. The supplied exact-surface baseline case-folds text, removes punctuation, and collapses spaces. Your conservative alternative will also use type, role/state, and neighborhood evidence. Relabeling and aggregating the occurrence projection under either mapping produces an **analysis-only reconciled projection**.

This distinction is central to the story. PANINI reconciles within a document but does not precompute these cross-document links. Its entity index returns the originating local node, and RICR discovers

a cross-document chain at read time. A global merge can turn homonyms, titles, years, occupations, or nationality values into false hubs; a conservative rule can miss aliases. Consequently, the reconciled projections below are used to learn about network sensitivity, never as retrieval evidence.

### 4.1 Question 2: Build the GSW network and reconcile entities [12 points]

The verified records from Question 1 are still hundreds of separate local workspaces. To study them as a network, first preserve exactly what was written inside each document, and only afterward ask whether two occurrences refer to the same real-world entity. Mixing those operations would make it impossible to tell whether a global edge came from a GSW or from your identity rule.

Construct the native directed multigraph by namespacing every node with its document and GSW filename. Add entity, space/time, and verb-phrase nodes. For each grounded QA answer, add a verb-phrase-to-answer edge keyed by QA ID and retain question text and provenance. Then project each local verb neighborhood onto its answer entities without merging across documents. Reconciliation must remain a separate occurrence-UID-to-global-ID mapping so that the same unreconciled graph can be evaluated under several identity hypotheses.

(a) Report native-graph node and edge counts by node/edge type and verify them against direct counts from the JSON files.

(b) Construct the weighted unreconciled entity projection. Verify that occurrence UIDs from different documents remain distinct, even when their names match.

(c) Run the exact-surface baseline, then implement a conservative reconciliation rule using surface, type/role compatibility, and local neighborhood evidence. State the decision rule precisely.

(d) Manually audit a fixed-seed sample of at least 30 proposed cross-document merges. Label each correct, incorrect, or uncertain; report estimated precision with uncertain cases shown separately. Include at least two false-merge and two missed-alias examples.

(e) Add unit tests for edge direction, provenance, projection weight, cross-document non-merge, intended merge, and homonym non-merge.

(f) Explain in your own words why entity reconciliation changes the mathematical graph rather than merely cleaning its labels. Use one false merge and one missed alias from your audit to support the explanation.

**Required output.** Construction and reconciliation code, the audit table, graph counts under all three identity conditions, passing tests, and a 200–300 word explanation.

**Point allocation.** Correct native graph (2), correct unreconciled projection (2), reconciliation method and audit (4), tests (2), and own-words explanation (2).

### 4.2 Question 3: Analyze the structured-memory network [12 points]

Question 2 produces several candidate "global memories," but network statistics can make a bad reconciliation look convincing. For example, merging every occurrence of a nationality, year, occupation, or ambiguous title can manufacture a giant component and high-centrality hubs that were never asserted by any document. Your task is therefore not merely to compute statistics; it is to decide which observed structure belongs to the local GSWs and which was created by the reconciliation rule.

Perform the following analysis separately for 2Wiki and MuSiQue. Compute every requested property on the unreconciled projection, exact-surface projection, and conservative projection; include native-graph measurements as a reference.

(a) Compute weakly connected components of the native graph and connected components of each entity projection. Report the number of components, giant-component size and fraction, isolated nodes, and component-size summary (minimum, median, mean, and maximum). Quantify how much each reconciliation rule changes these values.

(b) Plot degree probability mass and complementary cumulative degree distributions on log–log axes. State whether the heavy tail, if any, is consistent across datasets and reconciliation rules; do not claim a power law without a fitted test.

(c) For each reconciled projection, report the ten highest nodes under weighted degree, PageRank, and betweenness centrality. Audit whether hubs represent entities, common attribute values, or reconciliation errors. Exact betweenness or a documented fixed-seed approximation is acceptable.

(d) Report average clustering and degree assortativity, then visualize one complete gold multi-hop path from a development question with node types and edge/QA labels. Conclude with a specific explanation of why the reconciled projection must not be used by the PANINI retrieval pipeline.

(e) In your own words, distinguish a hub that reflects repeated semantic participation from a hub manufactured by reconciliation. Name the evidence you used to make that judgment rather than relying on degree alone.

**Required output.** At least four plots, two sensitivity tables, and a 200–300 word cross-dataset interpretation plus a 150–200 word hub explanation that separates local GSW structure from reconciliation artifacts.

**Point allocation.** Connectivity sensitivity (3), degree analysis (2), centrality audit (2), clustering/assortativity/path visualization (2), and own-words interpretation (3).

# 5 Stage III: A user asks a multi-hop question

The memory has now been inspected, but none of its global statistics answers a particular user. Suppose the question is, "Which film has the director who died later?" No single stored QA pair contains the final result. The system must first identify two film-to-director branches, retrieve two death dates, and compare them. This changes the computational object again: the next graph is not a memory graph but a **dependency graph of work still to be performed**.

The decomposer supplies a textual plan. Your parser turns that plan into retrieval nodes, deterministic-operation nodes, and dependency edges. Placeholders carry an answer forward, while independent branches may run in parallel. A malformed plan must be rejected before it can issue misleading retrieval calls.

## 5.1 Question 4: Question decomposition and dependency graphs [14 points]

The network audit does not yet answer a user's multi-hop question. RICR first turns that question into an executable plan. For example, "Which director died later?" may create two independent branches that retrieve each director and death date, followed by a deterministic comparison. The decomposer emits text, but the retrieval system needs a graph whose dependencies and operation types are explicit.

Run the supplied 4B decomposer on all 200 questions using the supplied prompt, sampling disabled, and the model-config maximum output length. Cache the raw output before parsing so a parser failure never requires another model run. Parse every numbered sub-question into a template node. A placeholder such as `<ENTITY_Q1>` creates an edge from Q1 to the dependent template. Reject duplicate IDs, missing or future references, cycles, and unresolved placeholders. Mark retrieval nodes separately from deterministic operations such as comparison or intersection, then topologically sort the result.

(a) Validate references: every placeholder must point to an earlier sub-question, the graph must be acyclic, and every retrieval node must contain a nonempty natural-language question.

(b) Identify independent linear sequences that may run in parallel and record the deterministic operations needed to combine their answers.

(c) On the public reviewed decomposition subset, report valid-plan rate, sub-question-count exact match, dependency-edge precision/recall/F1, and retrieval-vs-reasoning node accuracy.

(d) Analyze two errors from each dataset. Distinguish malformed output, wrong dependency, missing hop, extra hop, and semantically incorrect atomic question.

(e) Choose one valid multi-branch decomposition. Draw its dependency graph by hand, walk through its execution order, and explain in your own words why its final operation is reasoning rather than retrieval.

**Required output.** `decompositions.jsonl`, parser tests, a metric table, and four concise error analyses, plus one annotated hand-worked plan.

**Point allocation.** Inference and caching (2), parser/validation (3), parallel-sequence handling (2), metrics (2), error analysis (2), and own-words plan explanation (3).

# 6 Stage IV: Find one piece of evidence

The dependency graph produces an atomic question such as "Who directed *The Glass Wall*?" The system now needs an ordered list of GSW QA pairs that might answer it. Begin with lexical search so that every score can be explained from matched terms. Then add semantic search for paraphrases and entity-first search for questions whose best entry point is a named entity. All retrievers return the same record shape, `(qa_uid, score, rank, source)`, allowing their behavior to be compared without changing downstream code.

Unless a question explicitly asks for an ablation, use random seed 232, inner product over L2-normalized dense vectors, RRF constant 60, a pre-rerank pool of $M = 60$ unique QA records, and $k = 15$ returned candidates. Generation and model-based scoring are deterministic with sampling disabled. Retrieval expansion follows only document-local GSW adjacency; the reconciliation graphs from Stage II remain outside the operational pipeline.

## 6.1 Question 5: Sparse retrieval baselines [12 points]

The dependency graph now tells the system what atomic question to ask, but not where to find its answer. Begin with sparse methods because their behavior is transparent: when a result fails, you can inspect the matched terms and decide whether the problem is wording, entity description, or graph expansion. These methods also establish whether later neural components earn their additional cost.

Give every backend the same return format, `(qa_uid, score, rank, source)`, and build it first on a tiny hand-written corpus with one obvious answer. Using only the supplied metadata, implement and evaluate:

(a) TF–IDF retrieval over concatenated QA question and answer text;

(b) BM25 retrieval over the same QA text; and

(c) BM25 entity retrieval over surface name, aliases, roles, and states, followed by expansion to QA records attached to each retrieved entity.

(d) Select two atomic questions on which the methods disagree. Show the matched terms and scores, then explain in your own words how document frequency, BM25 length normalization, and entity expansion produced the different rankings.

Use identical text normalization within a method across datasets. State all tokenization and BM25 parameters. Evaluate the gold atomic sub-questions from the development splits at $k \in \{1, 5, 10, 15\}$. Report Recall@$k$ and MRR overall, by 2Wiki question type, and by MuSiQue hop count.

**Required output.** One implementation per method, a complete retrieval table, and a pair of annotated disagreements with a 200–300 word interpretation.

**Point allocation.** Correct TF–IDF/BM25 QA search (4), correct entity expansion (2), evaluation (2), and own-words retrieval interpretation (4).

### 6.2 Question 6: Dense, hybrid, and paper-style dual retrieval [16 points]

Sparse retrieval exposes a central difficulty of multi-hop search: an atomic question may paraphrase the stored QA text, while an entity mention may be the best doorway into a relevant local workspace. PANINI addresses these two failure modes with two independent routes—dense QA retrieval and sparse entity retrieval followed by local expansion—and joins them before reranking. Notice that this expansion uses document-local GSW adjacency from Question 2, never the analysis-only reconciled graph.

Load the supplied normalized QA embeddings and FAISS index; do not regenerate corpus embeddings. Use the supplied fixed-query embedding when present. If answer substitution creates a new query, encode it with the configured query encoder and cache it by exact normalized query string.

(a) Implement dense QA retrieval and verify that a manual inner-product calculation agrees with FAISS for at least five queries.

(b) Implement RRF over TF–IDF, BM25-QA, and dense rankings with constant 60. Deduplicate by stable QA ID.

(c) Implement paper-style dual retrieval: union QA records obtained from BM25 entity expansion and dense QA search, deduplicate to a pre-rerank pool of at most $M = 60$, rerank against the instantiated atomic question, and return the top $k = 15$.

(d) Compare all methods using the Question 5 protocol. Add mean and 95th-percentile retrieval latency and the average candidate count.

(e) For one query helped by dual retrieval, label every top-15 candidate by the route that introduced it. Explain in your own words why neither dense QA search nor entity expansion alone produced the same pool.

**Required output.** Search code, FAISS consistency test, performance/latency tables, the saved top-15 retrieval trace for every development atomic question, and one route-by-route explanation.

**Point allocation.** Dense retrieval (3), RRF (3), dual candidate construction (3), reranking/deduplication (2), evaluation (2), and own-words route explanation (3).

### 6.3 Question 7: Does reranking always help? [8 points]

Question 6 deliberately constructs a broad candidate pool: its goal is recall, so high-ranked items from different routes may be redundant or only superficially related. The reranker is supposed to repair that ordering using the instantiated atomic question. It can also overrule a strong retrieval signal and move the

first supporting QA downward. Treat reranking as a hypothesis to test, not an automatically beneficial step.

Freeze each query's $M = 60$ candidate pool so every method sees exactly the same QA IDs. Compare three scoring rules:

(i) reranker probability only;

(ii) reciprocal retrieval rank only; and

(iii) 0.5 reranker probability + 0.5 reciprocal retrieval rank.

Use rank reciprocal $1/r$ after assigning each candidate its best pre-rerank rank. Report Recall@15 and MRR for each dataset. Find one query for which reranking improves the first relevant rank and one for which it worsens it. For each, show the top five candidates before and after scoring and explain the lexical/entity mismatch. Then explain in your own words why a cross-encoder can be semantically more informed yet still make the ranking worse. Your answer must refer to the observed scores and candidate pool, not only to model size.

**Required output.** One comparison table and two annotated retrieval traces.

**Point allocation.** Controlled comparison (3), traces (2), and evidence-based own-words explanation (3).

# 7 Stage V: Turn retrieved pieces into a reasoning chain

Retrieving one QA pair is not enough for the original multi-hop question. After the first answer is selected, it is substituted into the next template, creating a new query that may lead to a different document. Every retained hypothesis therefore becomes a chain with its own evidence, instantiated questions, answers, and scores.

Use beam width $B = 5$ and $k = 15$ candidates per hop unless an ablation says otherwise. For a length-$t$ chain $C = (e_1, \ldots, e_t)$ with positive hop scores $s_1, \ldots, s_t$, score the whole path by

$$\text{score}(C) = \left( \prod_{\ell=1}^{t} \max(s_\ell, \epsilon) \right)^{1/t} = \exp\left( \frac{1}{t} \sum_{\ell=1}^{t} \log \max(s_\ell, \epsilon) \right), \qquad \epsilon = 10^{-12}. \tag{1}$$

Compute this geometric mean in the log domain. At every pruning point, sort by decreasing chain score and then by stable QA-ID sequence. Retain at most one chain for each normalized current answer. These rules make the search deterministic and prevent many differently worded copies of the same answer from occupying the beam.

## 7.1 Question 8: Implement Reasoning Inference Chain Retrieval [22 points]

At this point you have an executable decomposition and a retriever for one atomic question. The remaining problem is sequential: the answer selected at one hop changes the text of the next query. Greedily keeping only the highest-scoring first answer can destroy the correct chain, while retaining every candidate causes exponential growth. RICR resolves this tension with a small beam whose chains carry their own instantiated questions, evidence, and scores.

Complete `prune_unique_answers` and `run_linear_ricr` in `panini_course/ricr.py`. Represent a chain as an immutable ordered tuple of evidence records, current answer, instantiated questions, and hop scores. Implement the search first with a dictionary-backed toy retriever whose outputs you control, following this structure:

```
beams = retrieve(first_instantiated_question, k)
beams = prune_unique_answers(beams, B)
for template in remaining_templates:
    expansions = []
    for chain in beams:
        query = substitute(template, chain.current_answer)
        for candidate in retrieve(query, k):
            expansions.append(new_chain(chain, query, candidate))
    beams = prune_unique_answers(expansions, B)
```

For each linear sequence:

(1) retrieve $k$ candidates for the first instantiated sub-question;

(2) retain the top $B$ chains with distinct normalized current answers;

(3) substitute each current answer into the next dependent sub-question;

(4) expand all surviving chains by $k$ candidates;

(5) update the geometric-mean chain score in the log domain;

(6) globally prune the $kB$ expansions to $B$ distinct current answers;

(7) repeat until the sequence ends; and

(8) combine parallel sequences by deduplicating evidence using stable QA ID while preserving the highest-scoring provenance.

(9) Hand-work a two-hop example with $B = 2$ and $k = 2$, showing all expansions before and after global pruning.

(10) Explain in your own words why pruning separately inside each parent beam is not equivalent to global pruning, why unique-answer pruning promotes useful diversity, and why a geometric mean is used for chain scores.

A candidate from an earlier chain may not be mutated in place. Pruning is global across all $kB$ expansions, not performed separately per parent. Sort by decreasing geometric-mean score and then stable QA-ID sequence, retaining only the first chain for each normalized current answer. Ties must use the deterministic rule stated at the beginning of Stage V. Return both the deduplicated evidence and complete trace: instantiated questions, candidates, scores, pruned chains, and selected QA IDs at every hop. Only after toy cases pass should you connect the real retriever from Question 7.

**Required output.** Working implementation, all supplied RICR tests passing, at least three additional tests of your own (including parallel sequences), plus one human-readable trace from each dataset, the hand-worked beam, and a 200–300 word design explanation.

**Point allocation.** First-hop/pruning behavior (4), multi-hop expansion/substitution (5), chain scoring (3), parallel evidence merge (3), deterministic trace format (2), tests/edge cases (3), and own-words design explanation (2).

### 7.2 Question 9: RICR ablation study [14 points]

A working search procedure does not show that its design choices are necessary. A larger beam can preserve a correct early hypothesis but increases retrieval calls; unique-answer pruning buys diversity but may discard useful duplicate evidence; geometric means prevent long chains from winning merely because

they have more terms. Use controlled ablations to determine which mechanisms produce useful accuracy rather than accidental complexity.

Select a fixed, stratified 20-question subset from each development dataset and publish the IDs before running experiments. The default is $B = 5$, $k = 15$, unique-answer pruning enabled, geometric-mean scoring, and the best frozen Question 7 retrieval score.

Before running the experiments, write one directional prediction for each ablation below and justify it in your own words. Afterward, compare the observed result with the prediction and explain discrepancies using saved traces.

(a) Beam width: $B \in \{1, 3, 5\}$.

(b) Candidates per hop: $k \in \{5, 15\}$.

(c) Diversity: unique-answer pruning on versus off.

(d) Chain scoring: geometric mean versus last-hop score.

(e) Retrieval: BM25-QA, dense QA, RRF hybrid, and paper-style dual retrieval.

Report supporting-QA recall, complete-chain recovery, answer EM/F1, mean latency, and mean evidence count. Plot accuracy versus latency and accuracy versus evidence size. Change only the named factor in each comparison.

**Required output.** One prediction table, one ablation table, two trade-off plots, and a 300–400 word recommendation of the configuration you would deploy.

**Point allocation.** Experimental control and predictions (3), beam/candidate ablations (2), diversity/scoring ablations (3), retrieval ablation (2), trade-off analysis (2), and own-words causal interpretation (2).

# 8 Stage VI: Freeze the design and test whether it travels

The ablations end development. From this point onward, the system is treated as a finished design rather than a collection of tunable components. First it runs in its 2Wiki development domain; then the identical procedure is moved to MuSiQue, where longer chains test whether its behavior scales.

Use the provided normalization functions and macro-average over questions. For retrieval, report Recall@$k$ for $k \in \{1, 5, 10, 15\}$, reciprocal rank/MRR, supporting-document recall, and supporting-QA recall. For chains, report complete-chain recovery, average surviving chains, and average unique current answers. For final answers, report EM and token F1 using the best accepted alias. For efficiency, report latency per stage, peak GPU memory, reranked-candidate count, evidence count, and answer-context tokens. Every table states its evaluated-question count, and subgroup aggregates are computed from unrounded per-question values.

## 8.1 Question 10: End-to-end 2Wiki evaluation [10 points]

The component experiments now become one complete system in its development domain. Select the final configuration using only labeled 2Wiki development questions, record that choice, and freeze it before producing any held-out prediction. This separation matters: otherwise the final score measures continued tuning rather than generalization.

Run the frozen system on all 100 2Wiki questions. The answer model may see only the deduplicated QA evidence returned by RICR, never source documents, held-out labels, or reconciled-graph neighbors.

Its prompt must request `N/A` when the evidence is insufficient. Save the trace before generating the answer so retrieval and generation failures can be distinguished.

On the 80 labeled questions, report decomposition validity, supporting-QA recall, complete-chain recovery, EM, F1, average/95th-percentile latency, answer-context tokens, and evidence count. Break out EM/F1 by the four 2Wiki question types. Include one successful and one failed complete trace. For the failed trace, explain in your own words where the first irreversible error occurred and why later stages could not recover from it.

**Required output.** `results/2wiki_dev.jsonl`, `predictions/2wiki_heldout.jsonl`, one main table, one type table, and two traces.

**Point allocation.** Correct frozen run/output files (2), complete metrics (2), type-specific analysis (1), trace quality (2), grounded answer prompting (1), and own-words failure explanation (2).

### 8.2 Question 11: MuSiQue transfer and scaling evaluation [12 points]

The final test is whether the system learned a reusable procedure or a set of 2Wiki-specific choices. MuSiQue changes both the source distribution and chain length, including three- and four-hop questions. Treat it as a deployment to a new collection: package paths may change, but the algorithm and parameters may not.

Without changing retrieval weights, $M$, $k$, $B$, prompts, normalization, or score rules, replace the 2Wiki package with the MuSiQue package and run all 100 questions. On the 80 labeled questions:

(a) report the same metrics as Question 10, separated into 2-, 3-, and 4-hop groups;

(b) plot complete-chain recovery and F1 against hop count;

(c) compare 2Wiki and MuSiQue under the same frozen configuration; and

(d) identify whether the largest transfer loss comes from decomposition, first-hop retrieval, later-hop substitution/retrieval, or answer generation, using counts from saved traces rather than intuition.

(e) Explain in your own words why increasing hop count can affect complete-chain recovery differently from answer F1. Separate the effect of longer chains from other differences between the datasets.

**Required output.** Submit:

- `results/musique_dev.jsonl` and `predictions/musique_heldout.jsonl`;
- a hop-count table and two scaling plots; and
- a quantitative transfer-error analysis.

**Point allocation.** Frozen transfer run (2), hop-stratified metrics (2), scaling plots (2), cross-dataset comparison (2), traced error attribution (2), and own-words scaling explanation (2).

## 9 Stage VII: Hand the system to the next team

A research result is not finished when its final table is printed. The next person must be able to recreate the environment, resume expensive stages, trace a prediction to its evidence, and tell development outputs from held-out submissions. The project therefore ends where a real system handoff begins.

### 9.1 Question 12: Reproducibility and final submission [12 points]

Your final artifact should allow another student to reproduce a result from a fresh Colab runtime and to inspect where a wrong answer arose. Treat this as a handoff of the system rather than a notebook snapshot: paths must be relative, expensive stages restartable, configurations frozen in files, and predictions separate from gold labels.

Submit one archive or course-repository commit containing:

(a) `report.pdf`, organized by Questions 1–12 and limited to 18 pages excluding references and a two-page appendix;

(b) the completed Colab notebook and any Python modules/tests it imports;

(c) all four development/held-out JSONL files from Questions 10–11;

(d) `environment.txt` with Python, CUDA, package, and model revisions;

(e) a one-page "system story" in your report that follows one question from input through decomposition, retrieval, RICR, and final answer, explaining one design choice and one failure mode in your own words; and

(f) `RUNME.md` with exact commands, seeds, expected runtime, restart points, and the final Git commit hash.

The notebook must run from a fresh Colab runtime using relative paths after setting one `PACKAGE_ROOT`. Do not submit downloaded model weights, provided embeddings, FAISS indices, or dataset copies.

**Point allocation.** Report organization/completeness (3), executable code/tests (2), required result schemas (2), environment/RUNME reproducibility (2), and the own-words system story (3).

### 9.2 The prediction record passed between stages

Each prediction file must contain one JSON object per line and preserve the input order. Extra diagnostic fields are allowed, but the following fields are required:

```
{
  "dataset": "2wiki" | "musique",
  "split": "development" | "held_out",
  "question_id": "...",
  "question": "...",
  "predicted_decomposition": [...],
  "decomposition_valid": true,
  "retrieval_backend": "dual_hybrid",
  "beam_width": 5,
  "candidates_per_hop": 15,
  "chains": [
    {
      "qa_ids": ["...", "..."],
      "answers": ["...", "..."],
      "hop_scores": [0.91, 0.87],
      "chain_score": 0.8898
    }
  ],
  "evidence_qa_ids": ["...", "..."],
  "predicted_answer": "...",
  "latency_ms": {
    "decomposition": 0.0,
```

```
    "retrieval_ricr": 0.0,
    "answer": 0.0
  },
  "answer_context_tokens": 0
}
```

Development files may additionally contain gold labels and computed metrics. Held-out files must not contain gold fields.

## 10 Grading summary

| Component | Points |
|---|---|
| Q1. Understand the two data packages | 6 |
| Q2. GSW network and entity reconciliation | 12 |
| Q3. Structured-memory network analysis | 12 |
| Q4. Question decomposition and dependency graphs | 14 |
| Q5. Sparse retrieval baselines | 12 |
| Q6. Dense, hybrid, and dual retrieval | 16 |
| Q7. Reranking analysis | 8 |
| Q8. RICR implementation | 22 |
| Q9. RICR ablations | 14 |
| Q10. 2Wiki end-to-end evaluation | 10 |
| Q11. MuSiQue transfer and scaling | 12 |
| Q12. Reproducibility and submission | 12 |
| **Total** | **150** |

## 11 General grading and conduct

- A table or plot without axis labels, units, evaluated-question count, or a caption receives at most half credit for that item.
- Results that cannot be reproduced from the submitted code receive no implementation credit, even if the reported values are plausible.
- Clearly label any deviation from the required settings. An additional experiment does not replace a required one.
- You may use standard libraries for graph storage, TF–IDF, BM25, FAISS, plotting, and model loading. You may not import an implementation of RICR or copy an answer key.
- Follow the course collaboration and academic-integrity policy. Cite all external code, prompts, and written sources.
- The course late-submission policy applies. If a released artifact is corrupted or a Colab dependency becomes unavailable, post a minimal reproducer to the course forum before the deadline.

*The goal is not merely to obtain a final answer. Your system should make the network path, evidence, score, and failure mode inspectable.*